

RECALLOS

The Memory Layer for AI

Complete Setup and Implementation Guide

Version: 2.4

Released: 2026

Type: Professional Documentation

Persistent, Semantic, Model-Agnostic Memory for AI Agents and Applications

TABLE OF CONTENTS

1. Introduction and Overview	3
2. What is RecallOS?	4
3. The Problem It Solves	5
4. Core Architecture	6
5. Key Features and Benefits	7-8
6. Installation Guide	9-10
7. Quick Start Tutorial	11-12
8. Memory Collections	13
9. Semantic Search Capabilities	14
10. API Integration and Usage	15
11. Supported Integrations	16
12. Performance Benchmarks	17
13. Pricing and Plans	18
14. FAQ and Troubleshooting	19-20
15. Security and Compliance	21

1. Introduction and Overview

Welcome to RecalIOS, the memory infrastructure layer designed for modern AI applications. RecalIOS solves a fundamental challenge in AI development: persistent memory that survives across conversations, sessions, and deployments.

This comprehensive guide provides everything you need to understand, install, and deploy RecalIOS in your AI applications. Whether you're building chatbots, AI agents, copilots, or intelligent assistants, RecalIOS provides the memory infrastructure that keeps your AI context-aware and intelligent across multiple interactions.

Documentation Highlights:

• Complete installation and setup procedures
• Step-by-step configuration guides
• Integration examples with popular LLM providers
• Performance metrics and benchmarks
• Security and compliance information
• Troubleshooting and FAQ sections

2. What is RecallOS?

RecallOS is a production-grade memory infrastructure platform designed from the ground up for AI applications. It provides a unified memory layer that stores, indexes, and retrieves information with sub-50 millisecond latency.

Core Capabilities:

Feature	Description
Persistent Memory	Long-lived storage that survives across sessions, processes, and deployments
Semantic Search	Meaning-aware retrieval powered by hybrid vector and keyword indexes
Context Retrieval	Adaptive context windows tuned per model and per task
Project Memory Scoping	Organize and scope memory by project, customer, or environment
Agent Memory Tools	Drop-in tools your agents can call to recall, store, or forget
Cross-Session Recall	Pick up exactly where you or your agent left off
Memory Ranking	Importance and recency scoring keeps relevant facts accessible
Model Agnostic	Works seamlessly with Claude, GPT, Gemini, and open-source models

Technical Highlights:

- Sub-50ms retrieval latency optimized for production
- HNSW vector indexing for efficient similarity search
- Enterprise security with end-to-end encryption
- SOC 2 Type II compliant infrastructure
- Multi-region replication and point-in-time recovery
- 99.99% uptime SLA for production deployments

3. The Problem It Solves

The AI Amnesia Problem

Current AI systems suffer from a critical limitation: they forget. After each conversation, all context is lost. Projects lose continuity. Teams repeat themselves. Agents restart from zero.

Problem	Impact
Context Disappears Between Sessions	Users must re-explain context in every conversation
Projects Lose Continuity	Long-term projects lack coherent memory and history
Team Repetition	Teams waste time re-explaining decisions and context
Agent Amnesia	AI agents restart with zero knowledge, losing learned patterns
No Cross-Conversation Learning	AI cannot build cumulative knowledge over time
Context Window Limitations	Systems cannot leverage long-term learning patterns

RecallOS Solution: A unified memory layer that creates persistent, semantic memory across conversations and deployments. Your AI remembers everything.

4. Core Architecture

RecallOS uses a sophisticated, multi-stage pipeline that processes information from input through retrieval and back to your AI model. Every stage is instrumented, observable, and swappable for custom implementations.

The RecallOS Pipeline:

Stage	Component	Function
1	Signal Input	Receive user input or system signal
2	Memory Layer	Capture, route, and prioritize information
3	Embedding Engine	Encode semantic meaning from text
4	Vector Database	Index and store embeddings with metadata
5	Retrieval Layer	Rank and select relevant memories
6	AI Model	Reason, generate, and respond
7	Response Output	Deliver response back to user

Architecture Principles:

- Modular Design: Each stage can be customized or replaced
- Observable: Complete visibility into memory operations
- Efficient: Optimized latency throughout the pipeline
- Scalable: Handles millions of memories in production

5. Key Features and Benefits

RecallOS provides a comprehensive suite of features designed for production AI applications:

5.1 Persistent Memory

Store conversations, context, and knowledge permanently. Memories survive across sessions, restarts, and deployments. Never lose important context again.

5.2 Semantic Search

Goes beyond keyword matching. RecallOS understands meaning, context, and relationships between memories using hybrid vector + keyword indexing. Find relevant information even with different wording.

5.3 Adaptive Context Retrieval

Intelligently retrieves context tuned for each LLM and task. Different models have different optimal context windows. RecallOS adapts automatically.

5.4 Project-Scoped Memory

Organize memories by project, customer, or environment. Keep memories organized and isolated. Share memory across team members within a project.

5.5 Agent Memory Tools

Built-in tools for agents to `recall()`, `store()`, and `forget()` information. Agents can control their own memory lifecycle. Drop-in API for agent frameworks.

5.6 Cross-Session Continuity

AI systems pick up exactly where they left off. Maintain conversation history, decision context, and learned patterns across multiple sessions.

5.7 Memory Ranking

Intelligent ranking based on importance and recency. The most relevant facts float to the top. Stay focused on what matters.

5.8 Open Source

Run RecallOS locally, on your infrastructure, or as managed service. No vendor lock-in. Full control over your memory infrastructure.

5.9 Model Agnostic

Works with any LLM: Claude, ChatGPT, Gemini, DeepSeek, Qwen, Llama, Mistral, or custom models. Switch providers without losing a thought.

6. Installation Guide

RecalIOS can be installed and configured in minutes. Follow this step-by-step guide to get RecalIOS running in your environment.

6.1 System Requirements

Requirement	Specification
Operating System	Linux, macOS, or Windows (WSL2)
Python Version	3.8 or higher
Node.js Version	16.0.0 or higher (for CLI tools)
RAM	Minimum 4GB (8GB+ recommended)
Disk Space	20GB+ for vector indices
Database	PostgreSQL 12+ or SQLite 3.31+
Network	Internet connection for model API access

6.2 Installation Steps

Step 1: Install RecalIOS CLI

Install the RecalIOS command-line tool using pip or npm:

```
$ pip install recallos
$ recallos --version
```

Step 2: Initialize RecalIOS

Initialize your RecalIOS installation with configuration:

```
$ recallos init
? Database type: (PostgreSQL/SQLite) SQLite
? Memory location: /home/user/.recallos
? Enable vector indexing: Yes
? Embedding model: all-MiniLM-L6-v2
```

Step 3: Create First Memory Collection

Create a memory collection to store your first memories:

```
$ recallos memory create
? Collection name: my-project-memories
? Description: Project context and decisions
Collection created successfully!
```

Step 4: Verify Installation

Test your installation with a quick status check:

```
$ recallos status
Recallos v2.4.0
Database: Connected ✓
Embeddings: Ready ✓
Collections: 1
Total Memories: 0
```

7. Quick Start Tutorial

This section walks you through a complete end-to-end example of storing and retrieving memories.

7.1 Storing Your First Memory

Use the CLI to add a memory to your collection:

```
$ recallos memory add \ --collection my-project-memories \ --text "Company uses TypeScript for backend services" \ --tags "tech-stack,backend" \ --importance 8
```

The system will automatically create an embedding and store the memory with metadata.

7.2 Retrieving Memories with Semantic Search

Search for memories using natural language queries:

```
$ recallos memory search \ --collection my-project-memories \ --query "What programming language do we use?"
```

Output:

```
■■ Result 1: Similarity: 0.94
■ Company uses TypeScript for backend services
■ Importance: 8/10 | Tags: tech-stack, backend
■■ Created 5 minutes ago
```

7.3 Integration with Claude API

Here's a Python example integrating RecalIOS with Claude:

```
import anthropic
from recallos import RecalIOS

recall = RecalIOS(collection='my-project')
client = anthropic.Anthropic()

query = 'What tech stack do we use?'
relevant_memories = recall.search(query, limit=5)
context = '\n'.join([m.text for m in relevant_memories])

message = client.messages.create(
    model='claude-3-5-sonnet-20241022',
```

```
max_tokens=1024,  
messages=[  
{'role': 'user', 'content': f'{context}\n\n{query}'}  
]  
)  
print(message.content[0].text)
```

8. Memory Collections

Memory Collections are the primary organizational unit in Recallos. Each collection stores memories related to a specific project, customer, or context.

8.1 Creating Collections

```
$ recallos collection create \ --name customer-service \ --description 'Customer interaction memory' \ --retention-days 365
```

8.2 Collection Features

Feature	Description
Namespace Isolation	Memories in different collections are isolated
Access Control	Fine-grained permissions per collection
Retention Policies	Automatic cleanup of old memories
Metadata	Custom metadata per collection
Sharing	Share collections with team members

9. Semantic Search Capabilities

RecalLOS semantic search understands meaning, not just keywords. It uses advanced embedding models to find memories semantically similar to your query.

9.1 How Semantic Search Works

1. Your query is converted into a semantic embedding
2. RecalLOS searches the vector database for similar embeddings
3. Results are ranked by relevance and recency
4. Top results are returned with metadata

9.2 Hybrid Search Features

RecalLOS combines:

- **Vector Search:** Semantic similarity matching
- **Keyword Search:** Exact phrase matching
- **Metadata Filtering:** Filter by tags, dates, importance
- **Ranking:** Results ranked by relevance, recency, importance

9.3 Search Examples

```
Example 1: Basic semantic search
$ recallos memory search --query 'deployment strategy'
```

```
Example 2: Filtered search
$ recallos memory search \ --query 'performance optimization' \ --tags
'performance,optimization' \ --min-importance 7
```

10. API Integration and Usage

RecalLOS provides comprehensive APIs for programmatic access to memory functionality.

10.1 Core API Operations

Operation	Method	Description
Create Memory	POST /memories	Add new memory to collection
Search	GET /memories/search	Search memories by query
Retrieve	GET /memories/{id}	Get specific memory by ID
Update	PUT /memories/{id}	Update existing memory
Delete	DELETE /memories/{id}	Delete memory
List	GET /memories	List memories in collection

10.2 Python SDK Example

```
from recallos import RecalLOS, Memory

# Initialize client
recall = RecalLOS(api_key='your-api-key')
collection = recall.collection('my-project')

# Create memory
memory = collection.add(
    text='Database uses PostgreSQL with pgvector',
    tags=['database', 'infrastructure'],
    importance=9
)

# Search
results = collection.search('database technology')
for result in results:
    print(f"{result.text} (similarity: {result.similarity})")
```

11. Supported Integrations

RecallOS integrates seamlessly with major AI platforms and language models.

11.1 LLM Provider Integration

Provider	Model Family	Status
Anthropic	Claude 3 series	Connected ✓
OpenAI	GPT-4, GPT-3.5	Connected ✓
Google	Gemini Pro/Ultra	Connected ✓
DeepSeek	DeepSeek-67B	Available
Alibaba	Qwen series	Available
Meta	Llama 2/3	Available
Mistral	Mistral Large	Available

11.2 Integration Benefits

- **Model Agnostic:** Switch between providers without reconfiguring
- **Unified API:** Single interface for all model access
- **Automatic Optimization:** Context tuning per model
- **Cost Optimization:** Route queries to most cost-effective model

12. Performance Benchmarks

RecallOS is optimized for production deployments. Benchmarks below reflect real-world performance metrics from live operations.

12.1 Performance Metrics

Metric	v2.4	Benchmark Basis
Retrieval Speed (p95)	<40ms	1.2B tokens real workloads
Memory Accuracy	94%+	Quality evaluation
Context Coverage	98%+	Semantic relevance
Latency Overhead	<5ms	Per-request overhead
Token Savings	35-45%	vs. unlimited context

12.2 Production Reliability

Metric	Value
Uptime SLA	99.99%
Multi-Region Replication	Automatic
Point-in-Time Recovery	Up to 90 days
Per-Tenant Isolation	Default

13. Pricing and Plans

RecallOS offers flexible pricing scaled to your needs. Choose the plan that best fits your usage patterns and requirements.

13.1 Plan Comparison

Feature	Starter	Builder	Enterprise
Memory Entries	100k	10M	Unlimited
Monthly Cost	\$0	\$15/mo	Custom
Semantic Search	✓	✓	✓
Importance Ranking	×	✓	✓
Projects	1	Unlimited	Unlimited
Cross-Session Recall	×	✓	✓
Support	Community	Email	Dedicated

Starter Plan: Perfect for individual developers exploring RecallOS. Free tier with 100k memory entries and community support.

Builder Plan: Designed for teams shipping production agents. \$15/month billed annually. 10M entries, unlimited projects, priority support.

Enterprise Plan: Custom pricing for large-scale deployments. Unlimited memory, dedicated regions, SSO, on-prem options, 99.99% SLA.

14. FAQ and Troubleshooting

Q: How do I migrate from another memory system to RecalIOS?

RecalIOS provides migration scripts for common formats. Use the ``recallos migrate`` command to import memories from JSON, CSV, or other systems. See migration guide for details.

Q: Can I run RecalIOS locally without internet?

Yes, RecalIOS can run fully locally with SQLite and local embedding models. This doesn't require API keys or internet connectivity for memory operations.

Q: What happens if I exceed my memory entry limit?

On Starter plan, you'll receive warnings at 80% and 95% usage. At 100%, new memories trigger cleanup based on retention policy. Upgrade anytime to increase limits.

Q: How are memories encrypted?

RecalIOS uses AES-256 encryption at rest and TLS 1.3 in transit. Enterprise customers can enable end-to-end encryption with customer-managed keys.

Q: Can multiple users share memories in a collection?

Yes, collections can be shared with fine-grained permissions. Team members can have read, write, or admin access. Invite users with their email address.

Q: How often should I backup my memories?

Managed RecalIOS handles backups automatically with daily snapshots. For self-hosted, use ``recallos backup`` for manual backups or configure automated daily exports.

Q: What embedding model does RecalIOS use?

By default, RecalIOS uses all-MiniLM-L6-v2. You can configure different embedding models including OpenAI, Anthropic, or custom models via configuration.

15. Security and Compliance

RecalLOS is built with security as a core principle. We comply with industry standards and provide comprehensive security features for enterprise deployments.

15.1 Security Features

Feature	Description
Encryption at Rest	AES-256 encryption for all stored data
Encryption in Transit	TLS 1.3 for all network communications
Authentication	API key and OAuth 2.0 support
Access Control	Fine-grained RBAC and ABAC policies
Audit Logging	Complete audit trail of all operations
Data Isolation	Per-tenant data isolation by default
Compliance	SOC 2 Type II, GDPR, HIPAA-ready

15.2 Compliance and Certifications

- SOC 2 Type II certified infrastructure
- GDPR compliant with data deletion guarantees
- HIPAA-ready for healthcare applications
- CCPA compliant privacy controls
- Regular third-party security audits

15.3 Data Privacy

RecalLOS does not share customer memories with any third parties. All data remains under your control. Export or delete your memories anytime. For more details, see our [Privacy Policy](#) and [Terms of Service](#).

15.4 Reporting Security Issues

If you discover a security vulnerability, please email security@recallos.live with details. We appreciate responsible disclosure and will acknowledge receipt within 24 hours.

For more information, visit <https://www.recallos.live>

RecallOS is the memory layer for AI. Persistent, semantic, model-agnostic memory for agents, copilots, and assistants.